Abstract

This project focuses on designing an ECG sensor for capturing the electrical activity of heart. The ECG signals are then captured on arduino by the designed sensor and then serially transmitted to NI LabVIEW. In LabVIEW, the raw ECG signals are processed, including performing a Fast Fourier Transform (FFT) for frequency analysis, and are visualized on a waveform chart. Then the processed signal is then transmitted to Matlab via TCP/IP communication and then passed into a trained arhythmia classification model to be classified.

1.1 Introduction

Recently, real-time monitoring and analysis of biomedical signals have become increasingly important for the diagnosis and treatment of various health conditions. Among these signals, the electrocardiogram (ECG) is a crucial tool to assess heart health, as it records the electrical activity of the heart over time. An ECG is a test that records the electrical activity of the heart. An ECG sensor is used for purpose of recording the electrical activity of heart. NI LabVIEW is used for signal processing and signal conditioning and data measurng. Matlab is used to classify the ECG signal.

1.1.1 Objectives

- Design an ECG sensor to capture the electrical activity of heart.
- Send the captured ECG signal to the NI LabVIEW for processing.
- From LabVIEW transmit the ECG signal via TCP/IP communication to process it through a trained arhythmia classification model to classify the signal.

1.2 Equipments and Methodology

1.2.1 Equipment

- Arduino nano
- Electrodes
- TL-072
- NI LabVIEW
- Matlab
- AD8232 sensor

1.2.2 Methodology

- The AD8232 sensor was used to capture the heart signals from electrodes.
- Those signals were sent into instrumentation amplifier to take the difference between the signals from right arm and left arm while rejecting CMRR.
- From instrumentation amplifier the signal is sent into amplifier to be amplified and fed into arduino analog read pin.
- Arduino transmits the signal serially to LabVIEW.
- LabVIEW processes the signal and display the wavform in real time and takes FFT of the signal to measure the heartbeat.
- LabVIEW then transmits the ECG data to MATLAB via TCP/IP communication.

- Matlab then feeds the signal after processing it into the model and classify it using the trained Arhythmia Classification model.

1.3 Implementation And Results

1.3.1 Implementation

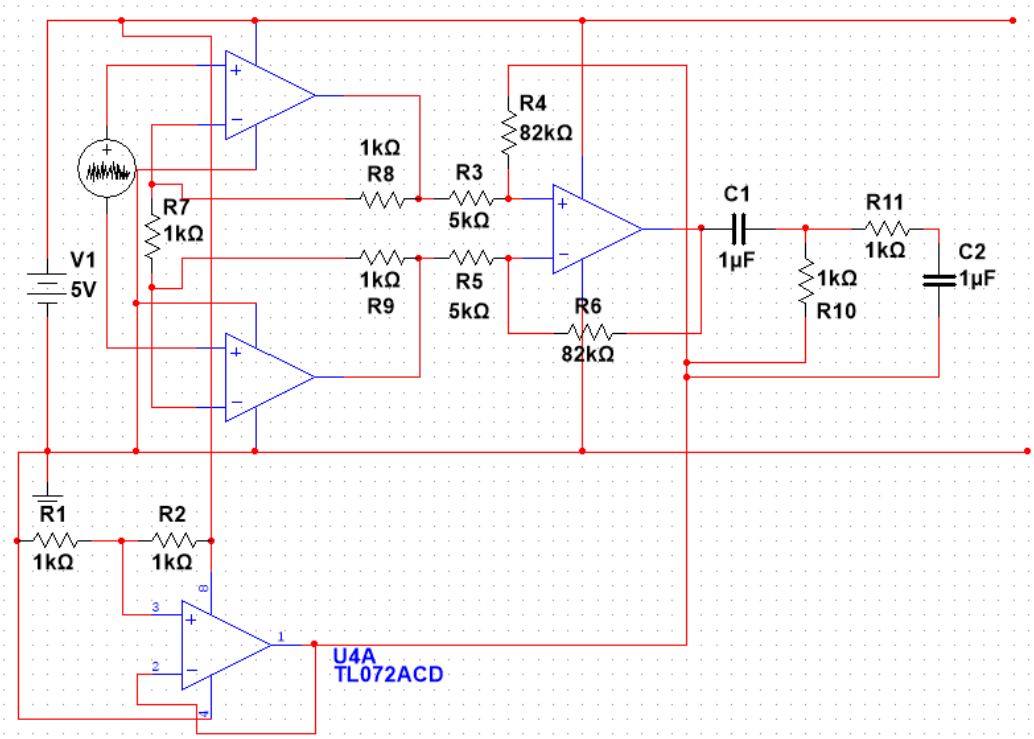

Figure 1.1: Circuit Diagram of ECG Sensor

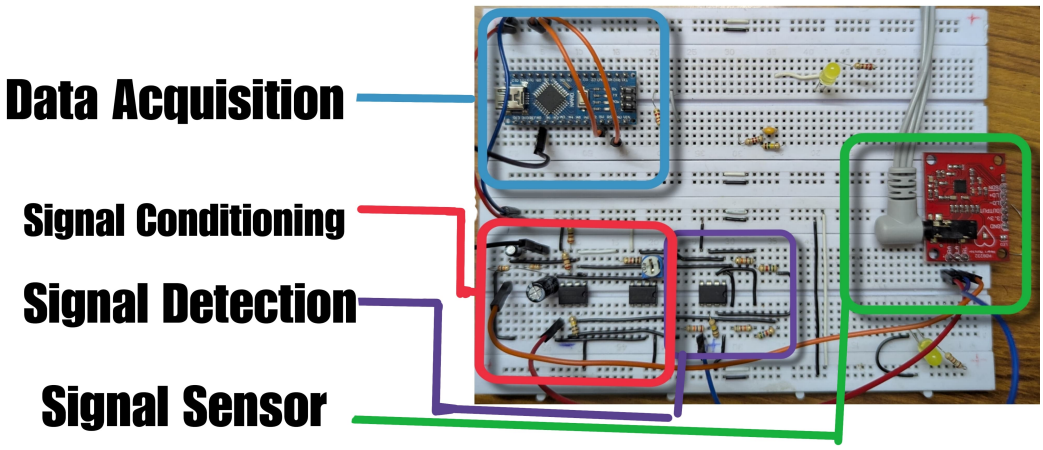

Figure 1.2: HardWare Implementation of ECG Sensor

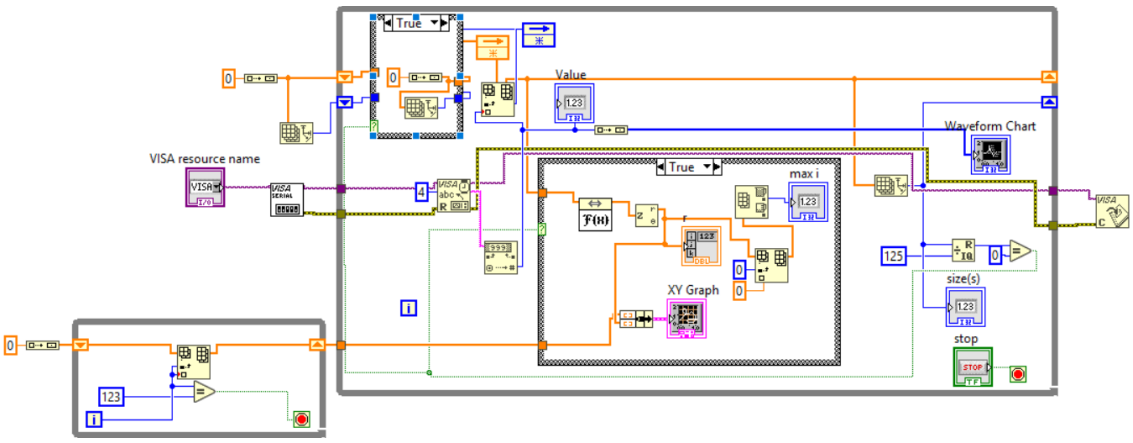

Figure 1.3: LabVIEW Implementation of Capturing ECG Data

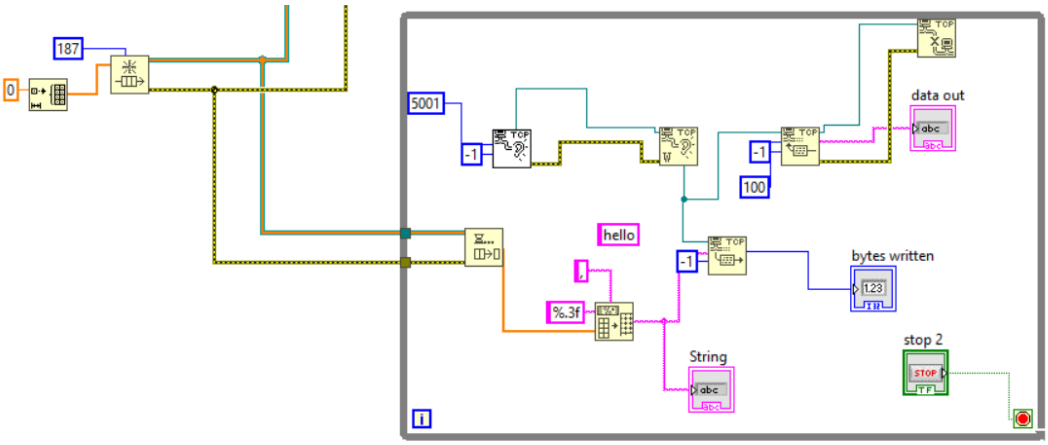

Figure 1.4: LabVIEW Implementation of TCP Communication with Matlab

1.3.2 Results

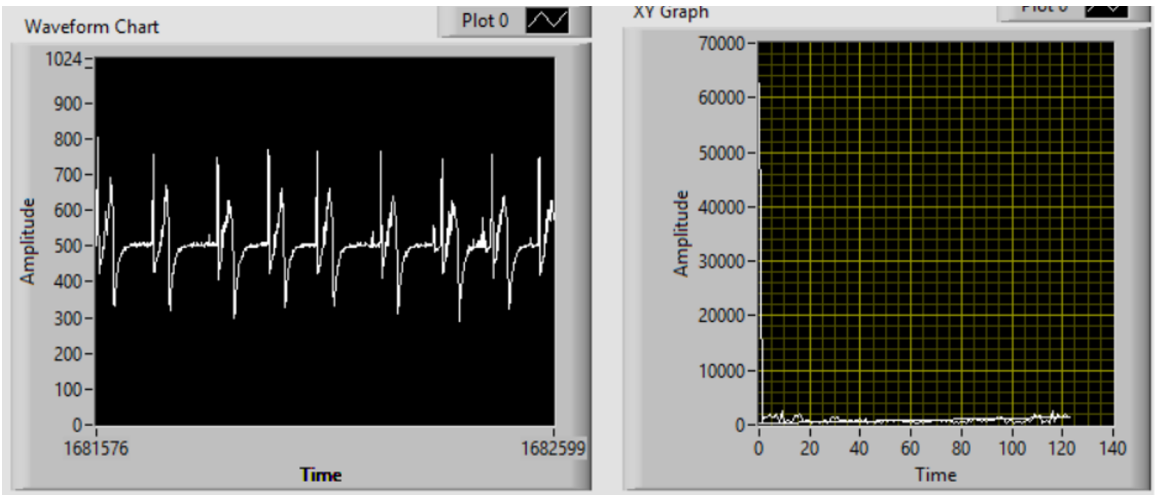

Figure 1.5: Real Time ECG Wavform and FFT

```
Connection Status: open
Data received: 0.900,0.851,0.577,0.147,0.000,0.114,0.200,0.209,0.316,0.349,


     The predicted class is:      2
```

Figure 1.6: Matlab ECG Signal Classification

1.4 Discussion

The ECG sensor was designed using the instrumentation amplifier which was used to amplify the ecg signal which is usually in the range of 5 mv and then signal was amplified and then fed into labVIEW serially. The signal Captured had alot of noise . This noise which was due to the muscle actions and respiratory movements was removed by the use of High Pass filter and electrical interferences were removed by using the Low Pass filters. Their was also ambiguity in the design as it sometimes

showed a promising Waveform but sometimes complete garbage values. LabVIEW software was also difficult to integrate since it does not properly integrate Matlab. The Mathscript in LabVIEW was not powerful enough to handle Machine learning models so had to go for TCP/IP communication with Matlab. We also used the labview software to capture the signal into an array, and take its Fast FOurier transform. THis allowed us to view the leading harmonic, and register it as the heartbeat of the person under test. This approach was quick, since it provided the frequency spectrum values and array of FFT magnitude, to display it on a simple graph.

The MATLAB network model, was written to process the array waveform data, with a CNN, and classify the patient into 4 categories, 2 of which being diseases,1 normal, and 1 unclassified. We trained this model over the data provided by kaggle.com, and tested its accuracy to be greater than 85 %. In the end, our own sensor's data was too noisy, for it to classify into a disease or normal class, but if used with a pre-built store bought ECG sensor, the results were satisfying, since it classified the data correctly.

1.5 Conclusion

In the end, the ECG sensor, was somewhat a success. And so was the MATLAB classifier, that we tested. But the integration of the two, into a single project that is a usable ECG, couldnt be done, since it was visibly, alright, but practically, it had a lot of noise in it. The major problem of the circuit was its ambiguity, and its issue with noise, and was a hardware failure. The software part of project played out successfully, as it was tested against unseen data.